SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1408 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I Ranjitha R(192421285) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ranji

Q. No	Problem Understanding(20)	Method Selection(20)	Solution Process(25)	Accuracy of Calculation (20)	Interpretation of Results(15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Q1. Smart Electricity Billing System

Concepts Covered: Intermediate Languages, Assignment Statements, Optimization

Problem Statement

An electricity billing system calculates the total payable amount using units consumed, rate per unit, and tax.

energy_cost = units × rate

tax_amount = energy_cost × tax_rate

total_bill = energy_cost + tax_amount

Tasks

(i) Write suitable declarations for all variables with appropriate data types.

(ii) Generate three-address intermediate code for the billing calculation.

(iii) Represent the intermediate code using quadruples, triples, and indirect triples.

(iv) Identify common subexpressions and optimization opportunities.

(v) Produce the optimized intermediate code with justification. (10 Marks)

Q2. Employee Promotion Evaluation System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

An employee is eligible for promotion if:

Experience ≥ 5 years

AND (Performance Score > 90 OR Certification = TRUE)

Tasks

(i) Formulate the Boolean expression.

(ii) Generate intermediate code using short-circuit evaluation.

(iii) Apply backpatching to resolve forward jumps.

(iv) Generate the final labelled intermediate code.

(v) Draw the control-flow graph for the evaluation process. (10 Marks)

Q3. Hospital Patient Priority System

Concepts Covered: Case Statements, Intermediate Languages

Problem Statement

A hospital assigns treatment priority based on emergency level.

```
switch(priority)

{

case 1: CRITICAL;

case 2: HIGH;

case 3: MEDIUM;

case 4: LOW;

}
```

Tasks

(i) Convert the switch statement into three-address code using conditional jumps.

(ii) Generate an equivalent jump-table implementation.

(iii) Represent the implementation using quadruples.

(iv) Compare conditional branching with jump-table execution.

(v) Recommend the most suitable implementation for emergency hospitals with justification. (10 Marks)

Q4. E-Commerce Payment Processing System

Concepts Covered: Procedure Calls

Problem Statement

An online shopping application processes payments through the following procedures.

amount = calculateAmount(cart)

processPayment(amount)

sendConfirmation(amount)

Tasks

(i) Generate intermediate code.

(ii) Illustrate parameter passing.

(iii) Design the activation record.

(iv) Draw stack diagrams for call and return operations.

(v) Explain how procedure calls are managed during execution. (10 Marks)

Q5. **Smart ATM Withdrawal Verification System**

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A customer can withdraw cash if:

Account Balance ≥ Withdrawal Amount

AND (PIN Verified = TRUE OR Biometric Verified = TRUE)

Tasks

(i) Write the Boolean expression.

(ii) Generate short-circuit intermediate code.

(iii) Apply backpatching.

(iv) Generate the final labelled intermediate code.

(v) Draw the control-flow graph. (10 Marks)

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Co_4

ASSESSMENT_2

NAME : RANJITHA R

REG NO: 192421285

COURSE CODE : CSA1408

COURSE NAME : COMPILER DESIGN

Q1. Smart Electricity Billing System:

Given:

energy_cost = unit × rate

tax_amount = energy_cost × tax rate

total_bill = energy_cost + tax amount.

i) Declaration:-

float units;

float rate;

float tax_rate;

float energy_cost;

float tax_amount;

float total_bill;

ii) Three ADD Code:-

T1 = unit * rate

T2 = T1 * tax_rate

T3 = T1 + T2

energy_cost = T1

tax_amount = T2

total_bill = T3

iii) Quadruples:-

No	operator	Arg 1	Arg 2	Result
1	*	units	rate	T_1
2	*	T_1	tax_rate	T_2
3	+	T_1	T_2	T_3
4	=	T_1	-	energy_cost
5	=	T_2	-	tax_amount
6	=	T_3	-	Total_bill

Triples :-

No	operator	Arg 1	Arg 2
1	*	units	rate
2	*	(0)	tax_rate
3	+	(0)	(1)
4	=	(0)	energy_cost
5	=	(1)	tax_amount
6	=	(2)	total_bill

Common subexpression / optimization :

There is no repeated subexpression in the gn. calculation

T_1 = units * rate

tax_amount = T_1 * tax_rate } unnecessary

total bill = T_1 + T_2 } recalculation avoided.

iv) optimized Intermediate Code :-

T_1 = units * rate

T_2 = T_1 * tax_rate

T_3 = T_1 + T_2

energy_cost = T_1

tax_amount = T_2

total_bill = T_3

Q2. Employee Promotion Evaluation System:-

i) Boolean Expression :-

Experience >= 5 AND (performance > 90 OR certification == TRUE)

Let,

E = Experience >= 5

P = performance > 90

C = certification == TRUE

Therefore,

E AND (P OR C)

ii) Short - Circuit Intermediate Code :-

if Experience < 5 goto L false

if performance > 90 goto L True

if certification == True goto L true

goto Lfalse

Ltrue :

Promotion = TRUE

goto Lend

L false:

Promotion = FALSE

Lend:

iii) Back patching :-

Experience < 5 → ?

Performance > 90 → ?

Certification = = TRUE → ?

After back patching :-

Experience < 5 → Lfalse

Performance > 90 → Ltrue

certification = = TRUE → Ltrue.

iv) Final Labelled Intermediate Code:

L_1 : if Exprevince < 5 goto L_4

L_2 : if performance > 90 goto L_3

L_3 : if certification ≠ = TRUE goto L_5

goto L_4

L_3 : promotion = TRUE

goto L6

L4 : Promotion = FALSE

L6 : End.

v) Control - Flow Graph :-

Experience > = 5 ?

Yes No

Performance > 90 ?

Yes No

Certification

TRUE ?

Yes No

Promotion

= TRUE

Promotion = FALSE

END

Q3. Hospital patient Priority System :-

i) Three Address Code using Conditional Jumps :-

if priority = = 1 goto L1

if priority = = 2 goto L2

if priority = = 3 goto L3

if priority = = 4 goto L4

goto L_5

L_1 : CRITICAL

goto L_5

L_2 : HIGH

goto L_5

L_3 : MEDIUM

goto L_5

L_4 : low

goto L_5

L_5 : END

ii) Equivalent Jump-Table Implementation :-

T_1 = priority - 1

goto Table [T_1]

Jump table:

table [0] → L_1

table [1] → L_2

table [2] → L_3

table [3] → L_4

Quadruples:-

No	operator	Arg 1	Arg 2	Result
1	==	Priority	1	L_1
2	==	Priority	2	L_2
3	==	Priority	3	L_3
4	==	Priority	4	L_4
5	-	Priority	1	T_1
6	goto	Table [T_1]	-	-

iv) Comparision:-

Conditional Branching	Jump Table.
uses multiple comparison	uses index directly
simple to implement	Faster for many cases
suitable for few cases	suitable for many consective cases.
More comparision may be required	Fewer comparision

v) Recommendation:-

Jump Table implementation is recommended for an emergency hospital system where there are many fixed and consective priority levels.

Q4. E-Commerce payment processing system:

The application performs:

amount = calculate amount (cart.

Process payment (amount)

Send confirmation (amount)

i) Intermediate Code:

Param cart

T_1 = call calculateAmount, 1

Param T_1

call processpayment, 1

Param T_2

call send confirmation, 1

amount = T_1

ii) parameter passing :-

call 1 → param cart

call calculate Amount

amount = calculate Amount (cart)

call 2 → param amount

call process payment

call 3 → param amount.

call send Conformation :-

iii) Activation Record :-

Parameters

↓

Return Value

↓

Return Address

↓

Control Link

↓

Local Variables

↓

Temproaries

iv) stack Diagram :-

Before call

Main program
cost
amount

During calculate Amount ()

Calculate Amount ()	← Top
cost	
Return address	
Local variable	
main program	
Cost	
amount.	

v) procedure Call Management :-

→ Parameter are passed to the procedure

→ An activation record created

→ Return address is stored

→ Procedure executes

→ Return value is produced

Q5. Smart ATM withdrawal Verification system.

i) Boolean Expression:

Balance >= withdrawal Amount

AND

(PIN_Verified == TRUE OR Biometric_Verified == TRUE)

B = Balance >= withdrawal Amount

P = PIN_verified == TRUE

Bo = Biometric - verified == TRUE

ii) Short Circuit Intermediate Code

If Balance < withdrawal Amount goto Lfalse

if PIN_verified == TRUE goto Ltrue

if Biometric_verified == TRUE goto Ltrue

goto Lfalse

Ltrue :

Dispense - Cash = TRUE

goto Lend

Lfalse :

Dispense _ cash = FALSE

Lend :

iii) Back patching :-

Balance < withdrawal Amount → ?

PIN _ Verified = = TRUE → ?

Biometric _ Verified = = TRUE → ?

Balance < withdrawal Amount → Lfalse

PIN _ Verified = = TRUE → Ltrue

Biometric _ Verified = = TRUE → Ltrue.

iv) Final Labelled Intermediate Code :-

L1 : if Balance < withdrawal Amount goto L5

L2 : if PIN _ Verified = = TRUE goto L4

L3 : if Biometric _ Verified = = TRUE goto L4

goto L5

L4 : Dispense _ Cash = TRUE

goto L6

L5 : Dispense _ Cash = FALSE

L6 : END

V) Control Flow Graph: